

Two Algorithms, Side by Side

Optimal Transport / Schrödinger Bridge Project

Companion to `nonuniform_grid_xy_variables.tex`, `nonuniform_grid_norms.tex`

Abstract

Just two listings, kept deliberately apart: (I) the LADMM iteration as it actually runs in `ladmm.py`, in pure math notation; (II) Goldstein–Li–Yuan’s PDHG (Algorithm 1, the plain iteration, and Algorithm 2, the adaptive step-size version), transcribed with every symbol that would otherwise collide with a Part I symbol renamed to something new – so a letter never means two different things across the two parts. No attempt yet to relate the two, substitute one into the other, or say which symbol plays which role – that’s later, once both sides are on the table and agreed.

Part I – my LADMM, as coded

Iterates x^k, y^k, δ^k ; scalars $\gamma_k > 0$ (penalty) and $\tau_k > 0$ (proximal step) – iteration-dependent, since the goal is to let them adapt (not yet wired up, but the iteration below is written for that from the start rather than as a fixed-step special case); linear/affine map A (boundary conditions baked in); convex functions f_1, f_2 . One step, $k \rightarrow k+1$, using γ_k, τ_k throughout (not γ_{k+1} or τ_{k+1} – those aren’t chosen until after this step, from its own residuals):

$$\begin{aligned} r^k &:= Ax^k - y^k \\ x^{k+1} &:= \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top (\gamma_k r^k - \delta^k) \right) \\ y^{k+1} &:= \text{prox}_{f_2/\gamma_k} \left(Ax^{k+1} - \delta^k / \gamma_k \right) \\ \delta^{k+1} &:= \delta^k - \gamma_k (Ax^{k+1} - y^{k+1}) \end{aligned}$$

where $\text{prox}_{th}(u) := \arg \min_p h(p) + \frac{1}{2t} \|p - u\|^2$. (Constant γ, τ is just $\gamma_k \equiv \gamma, \tau_k \equiv \tau$ – today’s actual behavior, and every formula below specializes to what was sent before when that holds.)

The x -update’s step, written out. In general $\text{prox}_{(1/\tau_k)f_1}$ genuinely depends on τ_k – this is the standard proximal-gradient step (step size $1/\tau_k$ on both the gradient move *and* the prox), and everywhere below is written to hold for that general case, since f_1 needn’t be an indicator for a generic LADMM problem. In *this* project’s own problem f_1 *is* the FP-feasibility indicator, so $\text{prox}_{(1/\tau_k)f_1}(v)$ collapses to the plain projection onto the feasible set for *any* τ_k (indicators’ proxes don’t depend on the step) – which is why earlier passes of this note wrote bare prox_{f_1} without losing anything *for this specific problem*. Kept general here regardless, so nothing below relies on that coincidence.

One simplification, flagged: `ladmm.py` also has an over-relaxation parameter α (in general, $\hat{z}^{k+1} := \alpha Ax^{k+1} + (1 - \alpha)y^k$ replaces Ax^{k+1} in the y - and δ -updates above). Every run in this project uses $\alpha = 1$, which is exactly the $\hat{z}^{k+1} = Ax^{k+1}$ substitution already made above – so this is the iteration actually being run, not a special case chosen for convenience.

Part II – Goldstein–Li–Yuan’s PDHG, renamed to avoid Part I’s letters

Renaming rule: keep their symbol unless it already means something else in Part I, in which case pick an unused letter. A, f_1, f_2^* turned out (Part III) to be the literal same objects as their operator and functions, not just analogues, so they’re written directly below rather than kept as separate placeholder symbols – only u, v (their x, y) need to stay distinct from Part I’s own x, y , which mean something else. Table first, algorithm after.

Their symbol	Clashes with Part I?	Used here instead
x (primal iterate)	yes – Part I’s x	u
y (dual iterate)	yes – Part I’s y	v
A (coupling operator)	no – same operator as Part I’s	A
F, G (convex functions)	no – $F = f_1, G = f_2^*$ exactly	f_1, f_2^*
τ_k (primal step)	yes – Part I’s τ	θ_k
σ_k (dual step)	no	σ_k (unchanged)
α_k (adaptivity level)	yes – Part I’s α	ω_k
Δ, η, s	no	unchanged
p_k, d_k (residuals)	no (D^k, P^k in <code>ladmm.py</code> are uppercase)	unchanged

Algorithm 1 (plain PDHG). Saddle point $\min_u \max_v f_1(u) + v^\top A u - f_2^*(v)$:

$$\begin{aligned} u^{k+1} &= \text{prox}_{\theta_k f_1}(u^k - \theta_k A^\top v^k) \\ v^{k+1} &= \text{prox}_{\sigma_k f_2^*}(v^k + \sigma_k A(2u^{k+1} - u^k)) \end{aligned}$$

Algorithm 2 (adaptive PDHG). After each Algorithm-1 step, form

$$\begin{aligned} p_{k+1} &= \left| \frac{1}{\theta_k}(u^k - u^{k+1}) - A^\top(v^k - v^{k+1}) \right| \\ d_{k+1} &= \left| \frac{1}{\sigma_k}(v^k - v^{k+1}) - A(u^k - u^{k+1}) \right| \end{aligned}$$

then

$$\begin{aligned} p_{k+1} > s\Delta d_{k+1} &\Rightarrow \theta_{k+1} = \frac{\theta_k}{1-\omega_k}, \sigma_{k+1} = \sigma_k(1-\omega_k), \omega_{k+1} = \eta\omega_k; \\ p_{k+1} < \frac{s}{\Delta}d_{k+1} &\Rightarrow \theta_{k+1} = \theta_k(1-\omega_k), \sigma_{k+1} = \frac{\sigma_k}{1-\omega_k}, \omega_{k+1} = \eta\omega_k; \end{aligned}$$

otherwise $\theta_{k+1} = \theta_k, \sigma_{k+1} = \sigma_k, \omega_{k+1} = \omega_k$. Required: $\sigma_0\theta_0 < \rho(A^\top A)^{-1}$. Their recommended defaults: $\omega_0 = 0.5, \Delta = 1.5, \eta = 0.95, \theta_0 = \sigma_0 = 1/\sqrt{\rho(A^\top A)}$.

Part III – relating them

The previous pass got stuck because Part II’s Algorithm 1 extrapolates the wrong variable to ever line up with Part I term-by-term. Fix suggested: reorder Algorithm 1 so the extrapolation is its own line, one step behind, built from a new variable w . That resolves it completely – not just “closely related,” an exact match.

Define, directly off Part I’s own x -update bracket – no elimination, this is literally `ladmm.py`’s `gamma*r - delta`, already computed every iteration:

$$w^k := \gamma_k r^k - \delta^k, \quad u^k := x^k, \quad v^k := -\delta^k.$$

Algorithm 1, reordered

$$u^{k+1} = \text{prox}_{\theta_k f_1} (u^k - \theta_k A^\top w^k) \quad (1)$$

$$v^{k+1} = \text{prox}_{\sigma_k f_2^*} (v^k + \sigma_k A u^{k+1}) \quad (2)$$

$$w^{k+1} = (1 + \rho_{k+1}) v^{k+1} - \rho_{k+1} v^k, \quad \rho_{k+1} := \sigma_{k+1} / \sigma_k \quad (3)$$

(3) is exactly $w^{k+1} = 2v^{k+1} - v^k$ – the proposed extrapolation – whenever σ_k is constant ($\rho_{k+1} = 1$); the ρ_{k+1} factor is only needed once σ_k genuinely varies, and falls out of the check below rather than being assumed.

Checking each line

(1). Pure substitution, no algebra: w^k 's definition dropped straight into Part I's x -update – step included, since $\text{prox}_{(1/\tau_k)f_1}$ genuinely depends on it in general,

$$x^{k+1} = \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top (\gamma_k r^k - \delta^k) \right) = \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top w^k \right) \stackrel{u=x, \theta_k=1/\tau_k}{=} \text{prox}_{\theta_k f_1} (u^k - \theta_k A^\top w^k) = ($$

(The $\theta_k = 1/\tau_k$ identification makes the *step*, not just the gradient coefficient, match too – $\text{prox}_{(1/\tau_k)f_1} = \text{prox}_{\theta_k f_1}$ exactly under that substitution, no separate argument needed.)

(2). Moreau's identity ($\text{prox}_{th}(u) := \arg \min_p h(p) + \frac{1}{2t} \|p - u\|^2$; verified numerically to machine precision in an earlier pass), for $h := f_2^*$, $t := \gamma_k$: $\text{prox}_{\gamma_k f_2^*}(u) = u - \gamma_k \text{prox}_{f_2/\gamma_k}(u/\gamma_k)$.

At $u := \gamma_k A x^{k+1} - \delta^k$:

$$\text{prox}_{\gamma_k f_2^*} (\gamma_k A x^{k+1} - \delta^k) = (\gamma_k A x^{k+1} - \delta^k) - \gamma_k \text{prox}_{f_2/\gamma_k} (A x^{k+1} - \delta^k / \gamma_k) \stackrel{\text{Part I } y\text{-update}}{=} \gamma_k (A x^{k+1} - y^{k+1}) - \delta^k = -\delta^k$$

i.e. $v^{k+1} = \text{prox}_{\gamma_k f_2^*} (v^k + \gamma_k A u^{k+1}) = (2)$ with $\sigma_k = \gamma_k$. ✓

(3). Straight from w 's definition at $k+1$: $w^{k+1} = \gamma_{k+1} r^{k+1} - \delta^{k+1}$. Part I's δ -update gives $\delta^{k+1} = \delta^k - \gamma_k r^{k+1}$, i.e. $r^{k+1} = (\delta^k - \delta^{k+1}) / \gamma_k$, so

$$w^{k+1} = \frac{\gamma_{k+1}}{\gamma_k} (\delta^k - \delta^{k+1}) - \delta^{k+1} = \rho_{k+1} \delta^k - (1 + \rho_{k+1}) \delta^{k+1} \stackrel{\delta=-v}{=} (1 + \rho_{k+1}) v^{k+1} - \rho_{k+1} v^k = (3). \quad \checkmark$$

(Starting at $k=0$: $w^0 := \gamma_0 r^0 - \delta^0$ needs no special convention, it's just the definition evaluated at $k=0$ – unlike the earlier attempt, no δ^{-1} or γ_{-1} bookkeeping is needed anywhere.)

An exact equivalence

$u^k = x^k, \quad v^k = -\delta^k, \quad w^k = \gamma_k r^k - \delta^k, \quad \theta_k = \frac{1}{\tau_k}, \quad \sigma_k = \gamma_k$

Part I (LADMM, as coded, γ_k, τ_k adaptive) is (1)–(3) – Algorithm 1, reordered – term for term, index for index, nothing left over. Algorithm 2's $\theta_{k+1}, \sigma_{k+1}$ update formulas (Part II) now translate directly into $\tau_{k+1} := 1/\theta_{k+1}$, $\gamma_{k+1} := \sigma_{k+1}$, and into (3)'s ρ_{k+1} – the one thing to carry along that their own paper doesn't need, since their un-reordered Algorithm 1 never has a second stored value of the dual variable to combine.

Where did y^k go?

Not lost – Moreau’s identity above only *needed* the difference $\gamma_k(Ax^{k+1} - y^{k+1}) - \delta^k$, but it was computed by literally evaluating $\text{prox}_{f_2/\gamma_k}(\cdot)$ along the way, and that inner evaluation *is* y^{k+1} (Part I’s own y -update, verbatim). Reading it back off, using $\text{prox}_{\gamma_k f_2^*}(u) = u - \gamma_k y^{k+1} = -\delta^{k+1}$ with $u = \gamma_k Ax^{k+1} - \delta^k$:

$$\gamma_k y^{k+1} = u + \delta^{k+1} = (\gamma_k Ax^{k+1} - \delta^k) + \delta^{k+1} \implies y^{k+1} = Ax^{k+1} + \frac{\delta^{k+1} - \delta^k}{\gamma_k},$$

and substituting $\delta = -v$:

$$\boxed{y^{k+1} = Au^{k+1} + \frac{v^k - v^{k+1}}{\gamma_k} = Au^{k+1} + \frac{v^k - v^{k+1}}{\sigma_k}} \quad (4)$$

(shift the index down by one for y^k in terms of iteration- $(k-1)$ quantities: $y^k = Au^k + (v^{k-1} - v^k)/\sigma_{k-1}$, valid $k \geq 1$). Sanity check against Part II’s own d_{k+1} residual (§ Part II): $d_{k+1} = |\frac{1}{\sigma_k}(v^k - v^{k+1}) - A(u^k - u^{k+1})|$ – visibly the same $\frac{1}{\sigma_k}(v^k - v^{k+1})$ term as in (4), but paired with $A(u^k - u^{k+1})$ there instead of Au^{k+1} here; they are related but not the same quantity, worth not conflating.

So: $x^k = u^k$, $\delta^k = -v^k$, $w^k = \gamma_k r^k - \delta^k$, $\tau_k = 1/\theta_k$, $\gamma_k = \sigma_k$, and y^k via (4) – every quantity in Part I now has an exact counterpart in Algorithm 1, reordered, and vice versa. What’s still open is not the equivalence itself but whether Algorithm 2’s p_k, d_k residuals and step-size update – derived and proved for their own (un-reordered) Algorithm 1 – remain valid (still drive θ_k, σ_k toward a convergent, stabilizing choice) once read against (1)–(3) instead. Not claimed either way here – next thing to check.

Part IV – can the extrapolation coefficient itself be fixed?

Part III’s equivalence is exact, but its extrapolation line needs a genuinely adaptive coefficient, $w^{k+1} = (1 + \rho_{k+1})v^{k+1} - \rho_{k+1}v^k$ with $\rho_{k+1} = \gamma_{k+1}/\gamma_k$ – not Algorithm 2’s own literal, fixed $(2, -1)$. Question: is there a reindexing that restores the fixed form exactly, for genuinely adaptive γ_k, τ_k ? Answer, as far as checked so far: yes, exactly one construction does it, and it comes at a real cost, not for free.

The fix: lag the x -update’s own step by one iteration

Leave the y - and δ -updates exactly as coded, using γ_k (governing the $k \rightarrow k+1$ transition, same as always). Change only the x -update, to use the *previous* iteration’s step sizes – *both* occurrences, the prox’s own step and the gradient coefficient, since (in general) they’re tied together and shifting one without the other would silently be a different, uncontrolled algorithm:

$$x^{k+1} := \text{prox}_{(1/\tau_{k-1})f_1} \left(x^k - \frac{1}{\tau_{k-1}} A^\top (\gamma_{k-1} r^k - \delta^k) \right) \quad (\text{was } \gamma_k, \tau_k - \text{see Part I}).$$

Define $w^k := \gamma_{k-1} r^k - \delta^k$, matching this new bracket. The *unshifted* δ -update already gives $\delta^k = \delta^{k-1} - \gamma_{k-1} r^k$, i.e. $\gamma_{k-1} r^k = \delta^{k-1} - \delta^k$ directly – no ratio factor, because both sides now refer to the same γ_{k-1} . So

$$w^k = (\delta^{k-1} - \delta^k) - \delta^k = \delta^{k-1} - 2\delta^k \stackrel{\delta=-v}{=} 2v^k - v^{k-1},$$

Algorithm 2’s literal, unmodified extrapolation – for *any* step-size sequence, adaptive or not, no special case needed.

The full correspondence, for this lagged version

Boundary convention first, spelled out rather than left implicit: w^0 needs γ_{-1}, τ_{-1} (there is no “iteration -1 ”), so set $\gamma_{-1} := \gamma_0, \tau_{-1} := \tau_0$ – the natural “no history yet, first step is unshifted” choice, giving $w^0 = \gamma_0 r^0 - \delta^0$ (Part III’s own w^0) and $v^{-1} := v^0$, so $w^0 = 2v^0 - v^{-1} = v^0$: no real extrapolation on the very first step, same as Part III.

With $u^k := x^k, v^k := -\delta^k$ as before, matching this lagged scheme against “Algorithm 1, reordered” ((1)–(3)’s template, now with the extrapolation fixed at $(2, -1)$) gives

$$u^k = x^k, \quad v^k = -\delta^k, \quad w^k = \gamma_{k-1} r^k - \delta^k, \quad \theta_k = \frac{1}{\tau_{k-1}}, \quad \sigma_k = \gamma_k$$

Look closely at the last two: σ_k uses γ_k directly (unshifted – the y, δ -update was never touched), but θ_k uses τ_{k-1} (shifted). That index mismatch is § “What this costs” point 1, made concrete: Algorithm 2’s own rule produces $\theta_{k+1}, \sigma_{k+1}$ as a *matched pair* from the same p_{k+1}, d_{k+1} , meant to govern the same transition – here $\sigma_{k+1} = \gamma_{k+1}$ governs producing δ^{k+2} from δ^{k+1} , while $\theta_{k+1} = 1/\tau_k$ doesn’t get used until producing x^{k+3} from x^{k+2} – one physical LADMM iteration later than its partner. Algorithm 2 was not written with that mismatch in mind; nothing here says it still behaves sensibly under it.

y^k ’s relation to u, v is untouched by any of this – it only ever depended on the (unshifted) y, δ -update, so Part III’s (4) carries over verbatim: $y^{k+1} = Au^{k+1} + (v^k - v^{k+1})/\sigma_k$.

Primal/dual residuals, adaptive step, lagged scheme

Derived directly from this problem’s own feasibility/stationarity conditions – not borrowed from `ladmm.py`’s or Goldstein–Li–Yuan’s own residual constructions. The Lagrangian sign convention that reproduces the already-established $-\delta^* \in \partial f_2(y^*)$ is $L_\gamma = f_1(x) + f_2(y) - \langle \delta, Ax - y \rangle + \frac{\gamma}{2} \|Ax - y\|^2$, giving

$$\text{primal feasibility: } Ax^* = y^*, \quad \text{dual feasibility: } A^\top \delta^* \in \partial f_1(x^*), \quad -\delta^* \in \partial f_2(y^*).$$

Primal residual: trivially $D^k := Ax^k - y^k - \text{ladmm.py}$ ’s own D (and, by this standard terminology, genuinely the *primal* one, reinforcing the earlier naming note).

Dual residual, y -side: exactly zero, always. y^{k+1} solves its subproblem exactly, and δ^{k+1} is defined so that its own optimality condition *is* $-\delta^{k+1} \in \partial f_2(y^{k+1})$ – not approximately, exactly, every iteration. All the real slack is on the x -side, where x is only linearized.

Dual residual, x -side. The lagged x -update’s own optimality condition, matching `ladmm.py`’s actual gradient bracket exactly:

$$\tau_{k-1}(x^k - x^{k+1}) - \gamma_{k-1} A^\top D^k + A^\top \delta^k \in \partial f_1(x^{k+1}).$$

Subtracting the true target $A^\top \delta^{k+1} \in \partial f_1(x^{k+1})$, and $\delta^k - \delta^{k+1} = \gamma_k D^{k+1}$:

$$P_{\text{lagged}}^{k+1} = \tau_{k-1}(x^k - x^{k+1}) + A^\top (\gamma_k D^{k+1} - \gamma_{k-1} D^k) \tag{5}$$

Directly traceable to this problem’s own feasibility conditions, line by line, using this scheme’s own updates – nothing borrowed. (Reduces, when γ, τ are constant, to $\tau(x^{k-1} - x^k) + \gamma A^\top (D^k - D^{k-1})$ after reindexing – close to but not identical to `ladmm.py`’s own coded $P^k = \tau(x^{k-1} - x^k) + \gamma A^\top (D^{k-1} - D^k)$, opposite sign on the second term. Not a contradiction: they’re different constructions from different proofs, both vanishing at a true fixed point; (5) is the one with a fully transparent derivation traceable to this problem’s own KKT conditions.)

Why the other direction doesn’t work

The mirror idea – leave the x -update alone, shift the y, δ -update *forward* to use γ_{k+1} instead of γ_k – gives the same clean algebra, but isn’t usable: γ_{k+1} , chosen (by Algorithm 2’s own rule) from residuals p_{k+1}, d_{k+1} , needs $v^{k+1} = -\delta^{k+1}$ – exactly the quantity that shifted δ -update is trying to produce. Circular. The backward shift above has no such problem: γ_{k-1} was already fixed, from residuals computed a full iteration earlier, well before the x -update at step k runs.

What this costs

Two things, both real, neither a labeling issue:

1. **An extra lag.** In Algorithm 2’s own convention, θ_j (decided from iteration- j residuals) is used immediately, to produce u^{j+1} . Here, τ_j (decided the same way, from the same residuals) isn’t used until producing x^{j+2} – one iteration later than Algorithm 2’s own rule assumes for its own step size. This is a structural change to the algorithm, not a renaming: τ ’s feedback loop is now one step slower than γ ’s.
2. **Still not literally their Algorithm 1.** This shifted scheme is exactly “Algorithm 1, reordered” (Part III) with a fixed extrapolation coefficient – but “reordered” extrapolates v into the u -update, while Goldstein–Li–Yuan’s own, published Algorithm 1 extrapolates u into the v -update. These are different recursions in general (not equal given the same u^k, v^k , except at a fixed point), so their convergence proof – and Algorithm 2’s proof that its adaptive rule stays convergent – was written for *their* ordering, not this one. Nothing here shows the shifted scheme converges under Algorithm 2’s step-size rule; it only shows that, *if* it converges, its extrapolation term has exactly the form they wrote down.

Where this leaves things

Of the two ways to try to reconcile adaptive γ_k, τ_k with Algorithm 2’s literal, fixed extrapolation, only one is even well-defined: lagging τ one iteration behind γ is causal; shifting the other way needs a residual before the quantity it depends on exists. So, as far as checked here: **this one-line, backward index shift on the x -update is the only way found so far to make adaptive-step LADMM and adaptive-step PDHG use the exact same, unmodified extrapolation formula.** It is not, by itself, a proof that the resulting scheme converges – that’s still open, and separate from Part III’s exact (but coefficient-adaptive) equivalence, which needs no such shift and no such open question.

Part V – what does the adaptive-LADMM literature actually do?

Checked (web search, not exhaustive) rather than assumed. Two representative papers that adapt a linearized-ADMM-style penalty parameter:

- Lin, Liu & Su, “Linearized Alternating Direction Method with Adaptive Penalty for Low-Rank Representation,” NeurIPS 2011 (arXiv:1109.0367) – LADMAP. Linearizes one subproblem exactly the way Part I does; adapts the penalty from a feasibility/residual condition. **No extrapolation or momentum term anywhere** – no $2x^{k+1} - x^k$, no lagged combination of past dual iterates. Just Part I’s update, run with γ_k adapted, unmodified otherwise.
- Xu, Figueiredo & Goldstein, “Adaptive ADMM with Spectral Penalty Parameter Selection,” AISTATS 2017 (arXiv:1605.07246) – penalty adapted each iteration via a Barzilai–Borwein spectral estimate from the primal/dual residuals. Every description found of it (full text wasn’t extractable to triple-check) is again a residual-driven penalty update on the standard ADMM/LADMM subproblems, with no mention of an extrapolation step.

Neither paper frames its adaptivity in terms of PDHG or Chambolle–Pock at all – the question of whether their adapted iteration still matches *some* adaptive PDHG scheme isn’t asked.

So, following the logic of Parts III–IV: both are exactly the “no lag” LADMM of Part I, with γ_k, τ_k adapted directly, no shift. By Part III, that recursion *is* Algorithm 1, reordered, with the *generalized* $(1 + \rho_{k+1}, -\rho_{k+1})$ extrapolation – not Goldstein–Li–Yuan’s own literal, fixed $(2, -1)$ Algorithm 2. So, as best determined here: **the adaptive-LADMM scheme actually used in this literature is a genuinely different algorithm from adaptive PDHG (Algorithm 2) whenever the step size actually changes between iterations, and coincides with it only in the constant-step case** ($\rho_k \equiv 1$, Part III’s original, pre-adaptive equivalence). That specific comparison – LADMM’s own common adaptive-penalty practice against Goldstein–Li–Yuan’s adaptive PDHG, specifically – wasn’t found stated anywhere in the two papers surveyed.

Closest related idea found, for honesty’s sake. Malitsky’s golden-ratio / reflected-gradient line of work (e.g. “Golden Ratio Algorithms for Variational Inequalities,” 2018/2020) does carefully track exactly this kind of issue in a related setting: their reflected/extrapolated schemes are proven equivalent to the relevant primal-dual method *under constant step sizes*, and their adaptive-step variants build the step size and the extrapolation from the same local history (consecutive iterates) rather than treating them independently – structurally the same flavor of fix as Part IV’s, though for a different family of methods (variational-inequality reflected gradient, not linearized ADMM specifically) and not verified here to reduce to Part III’s or IV’s formulas exactly. So the phenomenon itself – adaptive steps force the extrapolation/reflection term to track step-size history – isn’t unprecedented in the broader primal-dual literature; what wasn’t found is this specific comparison stated for LADMM’s own common adaptive-penalty practice against PDHG.

Part VI – dualizing the dual: a second route to Algorithm 1

A different idea for resolving Part III’s ordering mismatch: instead of forming the saddle point from the *primal* problem (Part II/III’s route), form it from the *dual* problem instead, and see if the resulting Algorithm 1 instance already matches Part I without any hand-built reordering. Worked through in full below – genuinely different from Part III’s route, not assumed to land

in the same place. New symbols throughout $(\tilde{u}, \tilde{v}, \tilde{\theta}, \tilde{\sigma})$, so nothing here is confused with Part II/III's own u, v, θ, σ .

The dual problem, and its own saddle point

Primal: $\min_u f_1(u) + f_2(Au)$, $u = x$ (Part II/III). Its Fenchel dual (general formula: $\min_x f(x) + g(Kx)$ has dual $\min_p f^*(-K^\top p) + g^*(p)$, derived earlier this session) is

$$\min_{\tilde{u}} f_2^*(\tilde{u}) + f_1^*(-A^\top \tilde{u}).$$

Legendre-transform *this* problem's own nonsmooth-composed piece $(f_1^*(-A^\top \tilde{u}))$, using $(f_1^*)^* = f_1$ to get a saddle point, matching Goldstein's template $(\min_X \max_V F(X) + \langle V, KX \rangle - G(V))$ with $X = \tilde{u}$, $F = f_2^*$, $K = -A^\top$, $G = f_1$:

$$\min_{\tilde{u}} \max_{\tilde{v}} f_2^*(\tilde{u}) + \langle \tilde{v}, -A^\top \tilde{u} \rangle - f_1(\tilde{v}).$$

Side by side

	Part II (primal-dualized)	Part VI (dual-redualized)
Saddle point	$\min_u \max_v f_1(u) + \langle v, Au \rangle - f_2^*(v)$	$\min_{\tilde{u}} \max_{\tilde{v}} f_2^*(\tilde{u}) + \langle \tilde{v}, -A^\top \tilde{u} \rangle - f_1(\tilde{v})$
X (no extrap., updates first)	u , role $F = f_1$	$\tilde{u}$, role $F = f_2^*$
V (uses extrap. fresh X)	v , role $G = f_2^*$	$\tilde{v}$, role $G = f_1$
Operator	A	$-A^\top$

Genuinely different from Part II's instance – which of f_1, f_2^* plays F vs. G is swapped, and the operator is $-A^\top$, not A . Algorithm 1's template, applied here:

$$\tilde{u}^{k+1} = \text{prox}_{\tilde{\theta}_k f_2^*}(\tilde{u}^k + \tilde{\theta}_k A \tilde{v}^k) \quad (6)$$

$$\tilde{v}^{k+1} = \text{prox}_{\tilde{\sigma}_k f_1}(\tilde{v}^k - \tilde{\sigma}_k A^\top (2\tilde{u}^{k+1} - \tilde{u}^k)) \quad (7)$$

Checking against Part I

(6), **the y/δ -side.** Try $\tilde{v}^k := x^{k+1}$ (forced – see below), $\tilde{u}^k := -\delta^k$: (6) reads $-\delta^{k+1} = \text{prox}_{\tilde{\theta}_k f_2^*}(-\delta^k + \tilde{\theta}_k A x^{k+1})$, which matches Part III's own y/δ -update (via the same Moreau step, unchanged) exactly with $\tilde{\theta}_k = \gamma_k$ – **no lag, no ρ -correction, clean.** This is a real improvement over Part IV's construction on this side.

(Why $\tilde{v}^k = x^{k+1}$, not x^k : Algorithm 1's own order updates X first, using whatever V is already stored – so (6) can only ever see a *stored* $\tilde{v}$, never a freshly-computed one. Part I's y/δ -update needs the *fresh* x^{k+1} (computed by the x -update earlier in the same LADMM iteration), so the only consistent identification is $\tilde{v}$ running one index ahead of x – forced by matching terms, not a free choice.)

(7), **the x -side.** With that same identification, $\tilde{v}^{k+1} = x^{k+2}$, and (7) becomes $x^{k+2} = \text{prox}_{\tilde{\sigma}_k f_1}(x^{k+1} + \tilde{\sigma}_k A^\top (2\delta^{k+1} - \delta^k))$. Matching against Part III's general (adaptive) x -update at the corresponding index forces $\tilde{\sigma}_k = \rho_{k+1}/\tau_{k+1}$ and $2\tilde{\sigma}_k = (1 + \rho_{k+1})/\tau_{k+1}$ simultaneously – solvable only when $\rho_{k+1} = 1$, i.e. $\gamma_{k+1} = \gamma_k$. Same verdict as Part IV, reached a different way: (7)'s extrapolation is fixed at $(2, -1)$ (built into Algorithm 1's template, unmodified, on

this side now), and fixed coefficients only reproduce fully adaptive-step LADMM when the step happens to be constant over that pair of iterations.

Worse, the fix isn't obviously free here either: making (7) exact for adaptive steps would need $\tilde{\sigma}_k = \rho_{k+1}/\tau_{k+1}$, i.e. knowing γ_{k+1} *before* computing $\tilde{v}^{k+1}$ – and γ_{k+1} would ordinarily be chosen from residuals of a transition this construction's own step order hasn't reached yet at that point. This has the same shape as Part IV's rejected "shift the other direction" argument, though I haven't nailed down with full certainty whether it's actually circular here or merely looks that way by analogy – flagged as open, not asserted.

Verdict

Not the same algorithm as Part II's, and not a way to escape Part III/IV's tension either – it relocates it. Genuinely useful outcome, though: it cleanly fixes *half* the mismatch (the y/δ -side needs no lag or correction at all, an improvement over every earlier construction), leaving only the x -side facing the same fixed-vs-adaptive-extrapolation issue Part IV already found. Reduces exactly to Part IV's constant-step case when $\gamma_k \equiv \gamma$. Worth having on record as a second, independent route to the same underlying obstacle – not a resolution of it.